

Scheduling in Time-Sensitive Networks Using Deep Reinforcement Learning



Abhilash Gopalakrishnan and Subhasri Duttagupta

Abstract Ethernet and wireless-based technologies play an important role in the Internet of Medical Things. The new healthcare scenarios require a deterministic communication system with real-time capabilities. In remote surgery, for example, the video latency is limited to 200 ms. In addition to latency, packet loss above a certain threshold can have a serious impact on the surgery. The majority of IP traffic today is served as best effort. Although assigning priority classes to flows can help, it is insufficient. A typical solution is to use high-speed media layers such as EtherCAT. However, these approaches do not integrate well with Ethernet. In this context, IEEE Time-Sensitive Networking (TSN) standards are gaining traction as a key enabler of real-time capabilities. TSN enables real-time communication by providing mechanisms for controlling latency, jitter, and packet loss. The purpose of this paper is to identify a scheduling strategy to minimize the violation of an end-to-end delay requirement while working with mixed-priority traffic, including time-critical ones. The majority of related work focus on static scheduling and cannot be learned continuously. This paper employs a deep reinforcement learning technique to enable dynamics and incremental learning. The scheduling problem is modeled as a Markov decision process, in which constraints are mapped to Rewards. In the proposed solution, learning converges in a few epochs and the results provide a feasible schedule. The schedule is then evaluated using simulation, which demonstrates that the deterministic requirements are met.

Keywords Time-sensitive networking · Ethernet · Reinforcement learning · Scheduling · Simulation

A. Gopalakrishnan (✉) · S. Duttagupta
Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, Kollam, India
e-mail: amenr4cse22012@am.students.amrita.edu

S. Duttagupta
e-mail: subhasrid@am.amrita.edu

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2024
S. M. Thampi et al. (eds.), *Applied Soft Computing and Communication Networks*,
Lecture Notes in Networks and Systems 966,
https://doi.org/10.1007/978-981-97-2004-0_13

193

1 Introduction

A wide variety of applications have been enabled by wireless and Ethernet technologies. These technologies play a significant role in the Internet of Things (IoT) paradigm. IoT is set to emerge as a major contributor in enhancing healthcare capabilities to reach more people. These scenarios in healthcare necessitate a deterministic communication system supporting real-time capabilities. A few examples include remote surgery and patient monitoring. In the case of remote surgery, the permitted latency of video is 200 ms beyond which it is unsafe to perform surgery. In the case of tactile feedback, permitted latency is as low as 10 ms. In patients' monitoring of vitals, the permitted latency is 250 ms [11]. In addition to latency considerations, loss of packets beyond a threshold could lead to poor Quality of Experience (QoE) leading to wrong interpretations and hence decisions. In order to handle deterministic/real-time scenarios like these, the typical approach has been the use of specific high-speed media layers. In industrial automation, this could mean the use of EtherCAT while for avionics it means Full-Duplex Switched Ethernet (AFDX). However, these approaches while satisfying their domain-specific requirements pose compatibility issues in converging with Ethernet.

The strategies to support QoS in Ethernet include a combination of scheduling protocols, prioritization, routing protocols, and resource management. A typical example is the VLAN priority tagging defined in IEEE 802.1 VLAN bridging standard. Many applications make use of VLAN tags and specify a priority class for achieving a certain amount of determinism. Publish-Subscribe protocols facilitate determinism by avoiding overheads, while reliability is ensured by sending the data more than once [7].

However, these are considered to be application-specific solutions and not enough to ensure QoS and determinism in emerging scenarios. It is in this context that standardization of real-time capabilities—IEEE Time-Sensitive Networking (TSN)—plays a major role. TSN focuses on mechanisms for ensuring real-time communication. One key aspect of the TSN standard is that it enables flows with mixed priorities and requirements. TSN achieves deterministic real-time communication, by controlling latency and jitter while ensuring reliability through zero loss. TSN uses time synchronization and scheduling across multiple network components. While TSN is reasonably mature over the Ethernet and in use in industrial applications, it is on the way to inclusion in wireless standards. An overview of TSN is key to the discussion.

TSN is a set of standards that enable determinism on a network as shown in Fig. 1. The standard focuses on ensuring a combination of zero congestion loss and bounded latency for time-critical defined traffic. It brings together varied traffic classes within the Ethernet network. TSN achieves this by combining mechanisms for per-traffic class scheduling and shaping, resource reservation, pre-emption, and replication. The scheduling and shaping aspects are defined in 802.1Qbv—Enhancements for Scheduled Traffic (TAS) and 802.1Qcr Asynchronous Traffic Shaping (ATS). Traffic shaping approaches can be time-scheduled, credit-based, or asynchronous. Resource

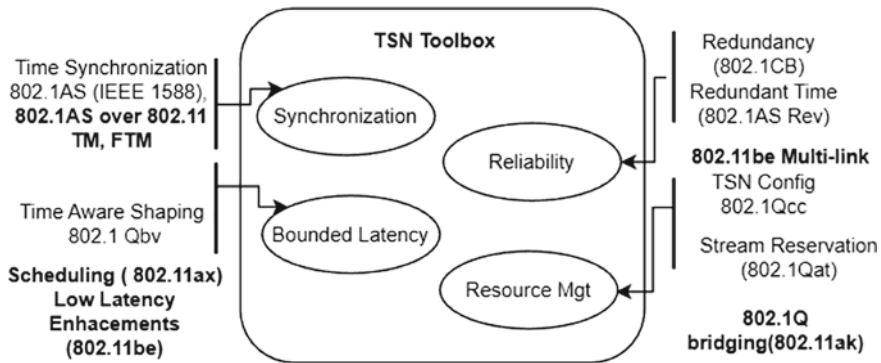


Fig. 1 TSN standards overview

reservation aspects are defined as part of 802.1Qcc—Stream Reservation Protocol and Frame Preemption defined in 802.1Qbu. Reliability aspects are defined in Frame Replication and Elimination—802.1CB.

Time synchronization over IEEE 1588 is a part of the standard as defined in 802.1QAS-Rev. Initial TSN standards were focused on Ethernet (IEEE 802.1) and built on VLAN Bridging (IEEE 802.1Q) [12]. Future wireless standards like Wi-Fi7 consider TSN as a key approach toward ensuring QoS.

As defined in 802.1Qbv each switch in TSN Network consists of Gate Control List and has gates which when open allow transmission of packets. As shown in Fig. 2, the TSN workflow begins with the user/administrator specifying the work-flows and their time criticality as well as priority considerations. This component which collects the specification is termed as Centralized User Configuration (CUC). The user configuration is now passed to a network controller. The Centralized Network Controller (CNC) uses this configuration coupled with topology to compute a schedule for communication flows. This computed schedule consists of the priority of the set of packets combined as flows and their start time, the timespan and in case of Time-Aware Shaper (TAS) specific timeslots where gates would open and close.

The purpose of this paper is to arrive at a scheduling strategy in order to minimize the violation of the end-to-end delay requirement while working with mixed-priority traffic including time-critical ones.

2 Related Work

A feasible schedule is one that satisfies the precedence, resource, and timing constraints. The objective of most scheduling approaches revolves around minimizing the schedule makespan while satisfying the constraints. As in Fig. 3 from an algorithmic approach point of view, the scheduling approaches can be categorized as based on Integer Linear Programming (ILP), constraint-based solvers like Satisfiability

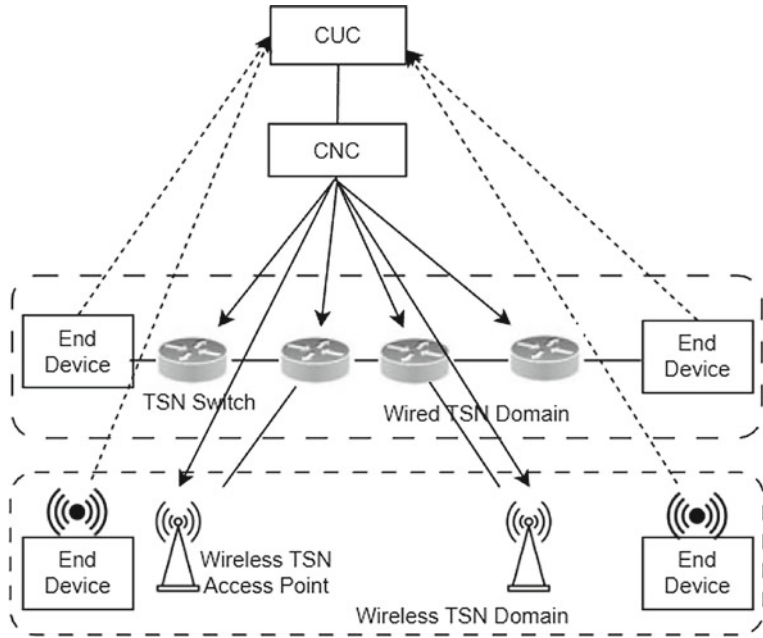


Fig. 2 TSN layout and configuration

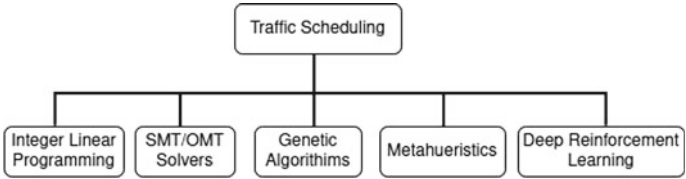


Fig. 3 Related work based on algorithmic approaches

ity Modulo Theory (SMT)/Object Modulo Theory (OMT), heuristic-based, genetic algorithms (GA), Metaheuristic, and deep reinforcement learning. Scheduling is often discussed in association with fixed/joint routing, mixed traffic, and reliability. Different approaches to traffic scheduling are presented in Fig. 3.

dos Santos et al. discuss time-aware schedule generation using solvers. They employ Satisfiability Modulo Theories (SMT) or Optimization Modulo Theories (OMT) solvers [3]. In this case, the latency and queuing requirements are translated into constraints and predicates and passed on to solvers like Microsoft Z3. These solvers analyze the schedules in terms of the constraints, taking into consideration the chances of a collision across flows. This analysis leads to identifying feasible schedules. The solvers work well for small-size networks with few endpoints and do not scale for large networks. Every time a schedule needs to be analyzed it needs to be sent to solvers, and hence this is static in nature. When the number of streams

involving complex topology is involved, the constraints-based approach grows exponentially. Scheduling being an NP-Hard problem heuristics is often considered as an approach to improve performance and capabilities. Heuristic Multiqueue Scheduler (HERMES) as discussed by Bujosa et al. uses multiple queues to support time-triggered traffic and provide high schedulability with very low scheduling times [1]. This approach is observed to perform better than constraint-based solutions and provides faster results.

Further improving on heuristics, Metaheuristics is considered an important option for scheduling for TSN. Min et al. discuss a method using Size Step Scheduling for Packet (PASS) to support no-wait time-aware scheduling. They experiment with multiple evolutionary methods like ant colony optimization and swarm approaches and compare their performance characteristics and reports. Of these approaches Artificial Ecosystem Optimization is found to achieve a lower cost [9].

One major advantage of having a framework like TSN is to be able to serve time-critical traffic while being able to balance best-effort traffic as well and ensure fairness. The challenges include ensuring fast response time and low latency. Mixed traffic further complicates the situation and necessitates efficient bandwidth management. In their work, Lu et al. discuss intelligent ultra-low latency communication [8]. The work proposes a method based on collision avoidance for deterministic scheduling and a dynamic scheduling method for non-time-sensitive flows, based on theoretical analysis of the combinability of flows.

Reliability is an important aspect of time-critical communication. Ensuring packets of critical communication are not lost is an important consideration in TSN Scheduling. Uncertainty is a characteristic of failures and errors. Feng et al. discuss scheduling from the context of reducing packet loss in their work [5]. Their work proposes a proactive fault-tolerant algorithm for TSN. One of the tradeoffs considered is schedulability, while the safety of critical flows is another. The wastage of network resources is a limitation of the work.

TSN considers a hybrid traffic communication system composed of both time-critical and best-effort traffic. The presence of mixed traffic makes it necessary to have routing effectiveness as well as scheduling effectiveness to ensure a deterministic and bounded delay. Since routing needs to go hand in hand with scheduling, a joint approach is often considered. This is an NP-Hard problem, being a combinatorial approach. Some of the work uses genetic algorithms. Pahlevan et al. discuss the usage of genetic algorithm (GA) in creating a population and validating it using fitness functions [10]. The GA approach considers the mapping of real-time applications and scheduling, multicast, and interdependence across flows while combining routing and scheduling. The results demonstrate significant improvement in schedulability due to the combinatorial approach and fares much better compared with the list scheduler which uses shortest route approaches. The fitness functions incorporate routing and scheduling in a single step and are faster. Some metaheuristic approaches using evolutionary algorithms have been tried out as well [9]. Moreover, wired and wireless integration and IoT scenarios leave us with more dynamics involved, and the environment turns out to be more stochastic/uncertain. With these dynamics, approaches using deep reinforcement learning are being considered.

In an IoMT scenario, the scheduling and routing would be dynamic considering the network is composed of wired, wireless, and ad hoc network scenarios. This is more stochastic in nature. Optimizing scheduling and routing with the help of deep reinforcement learning involves designing using the Markov decision process and reward modeling. The reward design needs to consider flows meeting the deadline as rewarded, flows not meeting the deadline as negatively rewarded, and lost packets also as negatively rewarded. Our objective is to ensure a balance between determinism and fairness and is able to ensure mixed traffic is served. The objective is to arrive at a policy for scheduling. The RL policy is a mapping between states and actions (in deterministic case) and the probability of actions (in stochastic case). The objective of optimal policy is to maximize the expected cumulative reward for every state. In reinforcement learning, we can use value-based approaches using Q value or policy gradient. The policy gradient method moves policy in the direction of the gradient (ascent) or actions that lead to more reward. Two major works are analyzed in detail.

Yang et al. suggest a graph convolutional neural network with deep reinforcement learning solution as a joint optimization problem in real-world communication scenarios [13]. The environment here considers both wired and wireless networks and multiple hops while also having a centralized plane or SDN and taking into consideration TSN. The method considers time-triggered, video and best-effort traffic. The solution involves providing a bounded worst end-to-end latency for lower-priority classes while guaranteeing the performance of high-priority traffic. The framework needs no prior knowledge of the system model and obtains information by interacting with the environment.

GCN is integrated with deep reinforcement learning as the function approximator to calculate the value function and strengthen the interrelationship. The convergence speed is accelerated by prioritized experience replay. Good convergence and better performance in terms of average end-to-end delay are observed from the solution. The evaluation included different network topologies and dynamic scenarios.

Chilukuri et al. propose RECCE, a deep reinforcement learning method in time-constrained networks for joint routing and scheduling in considering wireless and centralized control [2]. The focus of the work is scheduling optimal time slots for TDMA-based Medium Access Control. The delay at Medium Access Control is considered a major contributing factor. TDMA-based approach to the MAC layer is considered to bring determinism like in WirelessHART. Scheduling plays an important role in restricting end-to-end packet delay in multi-hop TDMA wireless networks. Optimal joint routing and scheduling for wireless networks with multiple hops is considered an NP-hard problem. Heuristic approaches are often used to improve the situation but lead to sub-optimal schedules. The main consideration is in meeting deadlines. In cases where the feasible schedule is not achievable, the approach tries to ensure the minimal number of packets missing the deadline. It also tries to reduce the time by which they miss deadlines. In the case of an available, feasible schedule, the focus is on minimizing average packet delay or maximum schedulability. The use of RL for joint routing and scheduling in wireless networks is considered novel. A single-agent approach with centralized scheduling

Table 1 Approach comparison

References	Highlights	Improvements and limitations
TSNsched [3]	Calculates static schedules using off-the-shelf schedulers. Time-critical requirements are translated to constraints	Leads to a schedule where collisions are avoided. Time complexity increases with the number of flows
HERMES [1]	Calculates a static schedule link by link. Heuristics used	Provides a schedule with low complexity
Metaheuristic [9]	A method called Packet Size Step scheduling for no-wait Time-Aware Shaper used. Various metaheuristic algorithms were evaluated	Optimization found as Artificial Ecosystem-based lowest cost
Genetic (GA) [10]	A flow schedule is represented as gnome to apply genetic algorithms for TSN scheduling. The fitness function incorporates deadline checks. Joint scheduling and routing	Can exponentially grow with the number of flows
GCNN DRL [13]	Applies graph neural networks as an approximation function to provide inputs to deep reinforcement learning. Considers TSN and uses replays to reduce computing. Joint scheduling and routing	Prepares effective schedules. Computationally intensive. Dynamic and can learn from prior baseline
RECCE [2]	Focuses on TDMA MAC layer delay handling. Deep reinforcement learning is used as a methodology. Joint scheduling and routing	Work well in wireless networks. Computationally intensive. Dynamic and can learn from prior baseline
RSC [6]	Suggests a non-work-conserving scheduler using Deficit Round Robin. A regulator is attached in front to limit the maximum burst	Despite lower complexity, it performs better than the current TSN approach

is used. Proximal policy optimization is used as the solution approach. Compared to other approaches, deep reinforcement learning approaches work well with stochastic effects and dynamics associated with networks. They can learn incrementally and evolve with the environment. A comparison of the most significant work in the table considering the technology, type of scheduling, and incremental learning is presented in Table 1.

It has been identified that schedulability improves by far when considering joint routing and scheduling. Integer linear programming-based approaches use linear equations to arrive at schedules. However does not scale. SMT/OMT Modulo theories analyze the collisions and, based on the constraints, provide a schedule using solvers. This approach is static and does not scale well. Genetic algorithm approaches create populations and perform mutations to arrive at an optimal schedule. These are static as well, involve computational effort, and do not scale. Heuristics and metaheuristic approaches work well in certain contexts but are static, do not scale well,

and produce non-optimal schedules. Deep reinforcement learning-based approaches are able to overcome these limitations as they accommodate dynamics well and can continuously learn and evolve.

The related work on scheduling has had a limited focus on mixed traffic flows. To the best of our knowledge, TSN-based time-critical networks over Ethernet serving mixed traffic with deadlines and priority considerations ensuring determinism and fairness have not been explored with reinforcement learning, which is the focus of our work. Furthermore, we follow a testbed-like approach, integrating validation over a simulation platform to check schedule effectiveness. We foresee that the approach of schedule generation using reinforcement learning when hosted on a network controller can help us continuously adapt to changes in the network.

3 The Solution Approach

The goal is to arrive at a feasible schedule that is optimal in terms of minimum average packet delay. In order to arrive at a solution using reinforcement learning, the scheduling problem needs to be modeled as a Markov decision process (MDP) as shown in Fig. 4. The agent needs to consider the number of nodes and the set of flows and define the state spaces and action spaces. The reward system needs to be designed to ensure the maximum number of packets reach the deadline while the minimum number of them is lost. The rewards could include a negative reward for missed packets and a negative reward for each packet lost. The objective is to maximize the number of packets delivered within a deadline, and the end-to-end delay is minimal. The methodology involves preparing an optimal schedule based on topology and flow specification and evaluating whether the schedule meets the deadlines as shown in Fig. 4.

The Network Model The network model is the network topology. A simple network consisting of three time-sensitive end system (ES) nodes ES1, ES2, and ES3 with two time-sensitive switches SW1 and SW2 is considered as shown in Fig. 5. The connections between these are represented as links.

The Traffic Model The traffic model is consisting of the flows and the initial sequence. The flows include high-priority flows of type video and best-effort flows. Four flows are considered Flow1, Flow2, Flow3, and Flow4, and they use the links Link1, Link2, Link3, and Link4 to pass from source to destination. The flows and links used are together represented in Table 2.

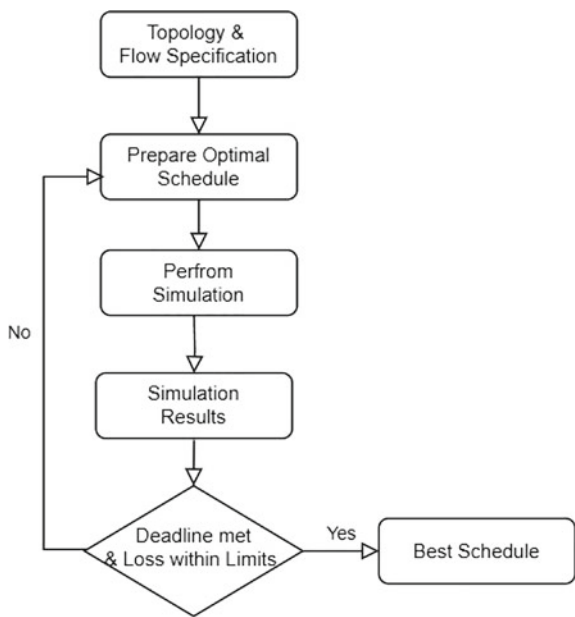


Fig. 4 Methodology followed

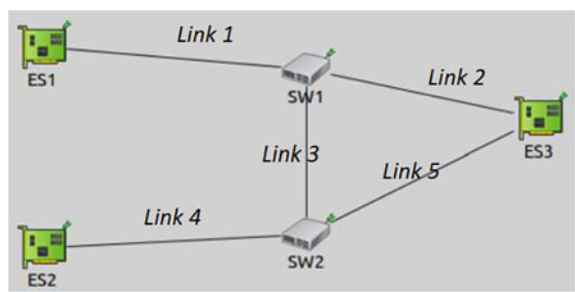
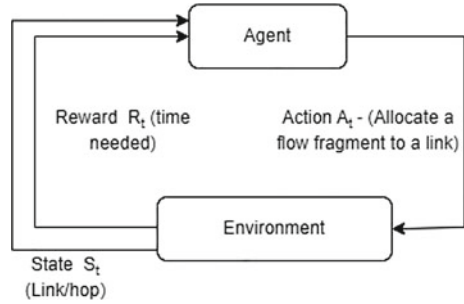


Fig. 5 Network topology

Table 2 Traffic flows and sequence

Flows	Links				
	Link1	Link2	Link3	Link4	Link5
Flow1	1	2	–	–	–
Flow2	1	–	2	–	3
Flow3	–	–	–	1	2
Flow4	–	3	2	1	–

Fig. 6 Overview of design for reinforcement learning



3.1 Reinforcement Learning

In order to apply reinforcement learning we need to define the Markov decision process (MDP). This requires us to define the elements of state spaces, action space, and reward design to be described in detail. An overview of MDP can be found in Fig. 6.

State Space The state space consists of the links or hops from one device to another. The state space consists of all links. Based on the simple network topology consisting of five links, the inputs state space is:

$$\{ES1 \rightarrow SW1, ES2 \rightarrow SW2, SW1 \rightarrow ES3, SW2 \rightarrow ES3, SW1 \rightarrow SW2\}$$

Action Space The action space consists of the allocating the flow fragments to the links or hops. The action space consists of allocating flow fragments

$$\{\text{Allocate Flow1 Fragment1} \rightarrow \text{Link1}, \text{Allocate Flow2 Fragment1} \rightarrow \text{Link2}, \dots\}$$

Reward Design The objective is to minimize the cumulative reward. In this case, the reward is re-scaled by dividing the makespan with the maximum operation length. There is a negative reward considered for not meeting the deadline and also for packet loss.

Setup and Environment The environment uses Open AI Gym and Ray Lib libraries. The NwSchedEnv is created using Open AI and integrates the state and action spaces for facilitating learning as in Fig. 7. The Ray Lib libraries serve and provide the agents and abstractions for Proximal policy optimization. The programming language used is Python. The environment is installed and then the application aspect is run. This initiates the training. The PPO approach builds on the Actor-Critic approach and uses Stochastic Policy Gradient. The Actor defined is using policy gradient which focuses on maximizing the reward, while the function approximation uses a neural network to learn the weights using TensorFlow. The algorithm details are described in Algorithm 1.

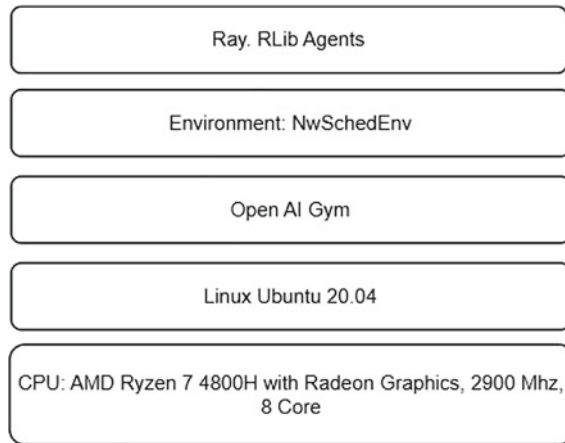


Fig. 7 Overview of environment for reinforcement learning

Algorithm 1 Schedule preparation algorithm

```

1: Input: Flow Specification with deadlines, Flow Sequence with Processing Time, Network Topology
2: Output: Predicted Schedule
3: Set State spaces as links.
4: Set Action spaces as link allocations.
5: Create Deep Q network model
6: Using PPOTrainer Train the model
7: Check if the links are available for allocation
8: Check for no operation case
9: if totalTimeofFlow > flowdeadline then
10:   reward -= timeNeededToComplete
11: else
12:   reward += timeNeededToComplete.
13: end if
14: Rationalize or Scale Reward.
15: Predict the schedule from the Model.
  
```

3.2 Simulation

The simulation tool used is OMNET++, a Discrete Event Simulator library and framework built using C++ and Qt framework libraries. Built on a kernel architecture style it supports communication network simulations using libraries like INET. OMNET enables a component architecture approach for models. Modules can be programmed using C++ and assembled. OMNET++ lets users to define topology using a Network Description (NED) file. Over and above these it lets users define the interfaces and build modules like the switches and access points extending current capabilities. OMNET++ also provides a Qt-based Graphical User Interface (GUI) to

help visualize simulations. OMNET++ has support for time-sensitive networking by means of libraries and is used by the IEEE standardization group as well [4].

As part of the simulation, the network model is described in OMNET++ as NED file. The schedule generated from the reinforcement learning environment is an array of start time slots, one array per flow. This is encoded to a JSON file and provided as input to TSN Scheduler in INET. On executing this simulation, it is possible to evaluate the effectiveness of the schedule and check if the determinism objectives are met.

4 Results and Discussions

The results are analyzed from the perspectives of scheduling using reinforcement learning perspective as well as scheduling effectiveness from time-sensitive perspective.

From a scheduling perspective using reinforcement learning, the focus is on a feasible schedule that does not overlap across the flows and evaluating the learning efficiency and convergence. No overlaps were found, and the schedule is within the deadline, thus pointing to a lower probability of packets missing deadlines. From a reinforcement learning perspective, the optimum schedule is achieved in 68 episodes, and then it stays stable, as found in Fig. 8. This includes negative reward considerations for crossing the deadline. Since a simple topology is considered to begin, the learning algorithm does this within 68 episodes. The training is computationally intensive, as we are using tensors and deep learning to obtain the weights.

In order to validate the time-sensitive nature of the schedule, it is compared with an Eager schedule approach. The different gate scheduler configurations used include Eager Scheduler which is a priority-driven scheduler, and TSN Gate Scheduler which goes by time slots and gates opening with the exact schedule defined. With the Eager Scheduler, we notice that the delay variations are high in the beginning, then they



Fig. 8 Results for reinforcement learning

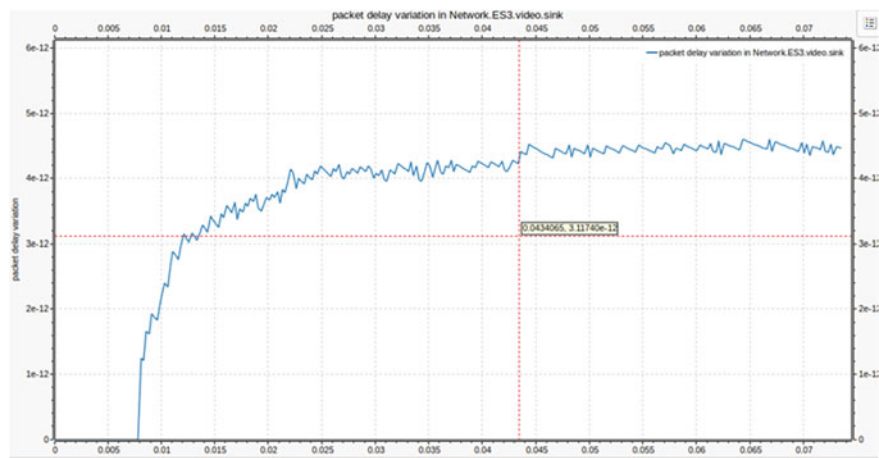


Fig. 9 Delay variation with Eager gate scheduling

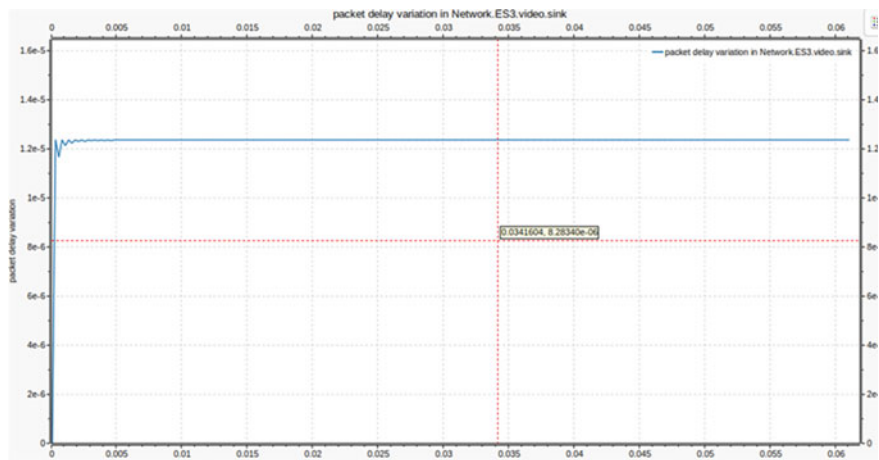


Fig. 10 Delay variation with TSN gate scheduling

improve, but it is still varying as in Fig. 9. Applying the TSN Scheduler using the RL-generated schedule the delay varies initially and stabilizes and stays as in Fig. 10. TSN Scheduler using the generated time slots is seen to provide a deterministic delay. The detailed result analysis can be found in Table 3.

Table 3 Results

Attribute	Description
Scheduling	The schedule prepared is found to be feasible schedule with no overlaps and a short makespan
Convergence	The schedule preparation training converges in a short period of 68 episodes
Delay variation	The delay variation obtained for Eager Scheduler is 2 times more than TSN Scheduler
Determinism objectives	For TSN Scheduler the delay stabilizes from 5 ms, while for Eager Scheduler even after 60 ms the delay continues to vary

5 Conclusion and Future Work

Scheduling in IoT networks requires a level of dynamics that can be well supported by deep reinforcement learning. The makespan obtained using Proximal policy optimization is more fine-tuned compared to results from genetic algorithms. Moreover, the approach is found to converge within a shorter time. The simulation results show that the generated schedule using DRL is both feasible and able to achieve deterministic objectives. In the next step, we plan to focus on building scenarios of scale, integrating wired and wireless with a focus on the network for healthcare scenarios.

References

1. Bujosa D, Ashjaei M, Papadopoulos AV, Nolte T, Proenza J (2022) HERMES: heuristic multi-queue scheduler for TSN time-triggered traffic with zero reception jitter capabilities. In: Proceedings of the 30th international conference on real-time networks and systems. RTNS 2022. Association for Computing Machinery, New York, NY, pp 70–80. <https://doi.org/10.1145/3534879.3534906>

2. Chilukuri S, Pesch D (2021) RECCE: deep reinforcement learning for joint routing and scheduling in time-constrained wireless networks. IEEE Access 9:132053–132063. <https://doi.org/10.1109/ACCESS.2021.3114967>

3. dos Santos ACT, Schneider B, Nigam V (2019) TSNsched: automated schedule generation for time sensitive networking. In: 2019 formal methods in computer aided design (FMCAD), pp 69–77. <https://ieeexplore.ieee.org/author/37087109659>

4. Falk J, Hellmanns D, Carabelli B, Nayak N, Dürr F, Kehrer S, Rothermel K (2019) NeSTiNg: simulating IEEE time-sensitive networking (TSN) in OMNeT++. In: 2019 international conference on networked systems (NetSys), pp 1–8. <https://doi.org/10.1109/NetSys.2019.8854500>

5. Feng Z, Cai M, Deng Q (2022) An efficient pro-active fault-tolerance scheduling of IEEE 802.1Qbv time-sensitive network. IEEE Internet Things J 9(16):14501–14510. <https://doi.org/10.1109/JIOT.2021.3118002>

6. Joung J (2019) Regulating scheduler (RSC): a novel solution for IEEE 802.1 time sensitive network (TSN). Electronics 8(2). <https://doi.org/10.3390/electronics8020189>. <https://www.mdpi.com/2079-9292/8/2/189>

7. León H, Montez C, Valle O, Vasques F (2019) Real-time analysis of time-critical messages in IEC 61850 electrical substation communication systems. *Energies* 12(12). <https://doi.org/10.3390/en12122272>. <https://www.mdpi.com/1996-1073/12/12/2272>
8. Lu Y, Yang L, Yang SX, Hua Q, Sangaiah AK, Guo T, Yu K (2023) An intelligent deterministic scheduling method for ultralow latency communication in edge enabled industrial internet of things. *IEEE Trans Ind Inform* 19(2):1756–1767. <https://doi.org/10.1109/TII.2022.3186891>
9. Min J, Oh M, Kim W, Seo H, Paek J (2022) Evaluation of metaheuristic algorithms for TAS scheduling in time-sensitive networking. In: 2022 13th international conference on information and communication technology convergence (ICTC), pp 809–812. <https://doi.org/10.1109/ICTC55196.2022.9952760>
10. Pahlevan M, Obermaisser R (2018) Genetic algorithm for scheduling time-triggered traffic in time-sensitive networks. In: 2018 IEEE 23rd international conference on emerging technologies and factory automation (ETFA), vol 1, pp 337–344. <https://doi.org/10.1109/ETFA.2018.8502515>
11. Qureshi HN, Manalastas M, Ijaz A, Imran A, Liu Y, Al Kalaa MO (2022) Communication requirements in 5G-enabled healthcare applications: review and considerations. *Healthcare* 11(6). <https://doi.org/10.3390/healthcare10020293>
12. Seol Y, Hyeon D, Min J, Kim M, Paek J (2021) Timely survey of time-sensitive networking: past and future directions. *IEEE Access* 9:142506–142527. <https://doi.org/10.1109/ACCESS.2021.3120769>
13. Yang L, Wei Y, Yu FR, Han Z (2022) Joint routing and scheduling optimization in time-sensitive networks using graph-convolutional-network-based deep reinforcement learning. *IEEE Internet Things J* 9(23):23981–23994. <https://doi.org/10.1109/JIOT.2022.3188826>